CLI-First: What an Agent Knows About an Interface Matters More Than Its Protocol

Chandra Irugalbandara
Leaf Monkey Labs
Colombo, Sri Lanka
chandra@leafmonkey.org

MCP, curated server (MCP-24)

```
get_trial_balance()
get_accounts()
create_reminder("Review spending: General Expenses", "2026-04-30")
create_reminder("Review spending: Fuel", "2026-04-30")
create_reminder("Review spending: Pharmacy", "2026-04-30")
… 21 more create_reminder calls
```

3 passes 26 tool calls 4.1k output tokens **11.7¢** 25 s

CLI with the command sheet, composed (CLI sheet/comp)

```
# call 1
salli accounts list --json > $TMPDIR/a.json 2>/dev/null
salli ledger trial-balance --json > $TMPDIR/tb.json
jq -r --slurpfile tb $TMPDIR/tb.json '.[]
  | select(.type=="expense") | [.id,.name,.currency,
    ($tb[0][.id]//"none")] | @tsv' $TMPDIR/a.json
# call 2
jq -r --slurpfile tb $TMPDIR/tb.json '.[]
  | select(.type=="expense" and .currency=="LKR")
  | select((($tb[0][.id]//"0")|tonumber)>20000)
  | .name' $TMPDIR/a.json |
while IFS= read -r n; do salli reminders add \
  "Review spending: $n" --due-date 2026-04-30 --json; done
```

3 passes 2 tool calls 0.9k output tokens **4.4¢** 48 s

Figure 1: The same task through two interfaces to one product. Asked to set a review reminder for every expense account over LKR 20,000, an agent with Salli's MCP server called a tool 26 times; an agent given the CLI's command sheet wrote a two-call script ending in one loop, produced a fifth of the output tokens and cost 62% less, but took twice as long and put 24 writes behind one tool call. This is the cell where composition saved most; over both repetition tasks the saving was 36–39%, and on chains CLI agents without the command sheet cost more than MCP (Section 6.5). Medians of three Sonnet runs on the large ledger. The left panel abridges an actual trace; the right reproduces one run's two commands, with line breaks added.

ABSTRACT

Coding agents now build and operate much of the software they touch, mostly from a terminal. We propose CLI-first development: a capability is done only when it works from the product's command-line interface, and that CLI is the reference client against which the product's other clients are checked. In a personal-finance product that adopted the rule, CLI coverage fell behind the product's API until a dedicated phase restored it, and a coding agent's pass through the CLI found a safety control that held on the web path but not on the CLI's. We then test a popular claim, that CLIs cost agents less than Model Context Protocol (MCP) servers. Holding the product fixed, 704 runs crossed the protocol, what the agent knows about the interface, and whether it may compose operations. What the agent knew mattered more than the protocol. Agents that learned the CLI from its help cost 1.6 to 2 times as much as agents given the product's curated MCP server. A compact command sheet brought the CLI level with that server on chains of dependent steps for the stronger model, though not the weaker. With the sheet, agents replaced 26 tool calls with a two-call script and cost 36 to 39% less on two tasks that repeat an operation, at the price of coarser oversight. The curated server was the fastest configuration.

CCS CONCEPTS

• **Human-centered computing** → **Interaction paradigms**; • **Software and its engineering** → *Software verification and validation*; • **Computing methodologies** → *Intelligent agents.*

KEYWORDS

AI agents, command-line interfaces, Model Context Protocol, tool use, agent-computer interfaces, cost, oversight

1 INTRODUCTION

Agents are becoming a large share of the users of developer tools. Cloudflare reports that agents accounted for a quarter of the use of its command-line tool in March 2026, up from single digits a year earlier, and for 48% by late September [12]. Inside product teams the shift shows in version control: in Salli, the personal-finance product we study, 75% of commits carry a coding agent's co-authorship trailer. Those agents need to run the software they change, not just compile it.

We propose *CLI-first development*, a process with two rules: a capability is done only when it works from the product's command-line interface (CLI), and the CLI is the *reference client* that the product's other clients, including the web application and any Model Context Protocol (MCP) server, are checked against. The proposal makes two claims. One concerns process: a CLI held to these rules lets the agents that build a product exercise and verify all of it. We examine it in Salli's version history. The other concerns cost and is widely argued, while the few measurements disagree: that a CLI is cheaper for agents than an MCP server because MCP loads every tool definition into the model's context [51, 54], or more expensive because every call starts a process [36].

To test the cost claim we held one product fixed and varied only the interface its agents used (Figure 1), crossing three factors: the *protocol* (MCP tool calls or shell commands), what the agent *knows*

about the interface before it starts, and whether it may *compose* operations into one program. If the interface did not matter, every configuration would cost what Salli's curated MCP server costs, and every ratio we report would be 1. What the agent knew moved cost more than the protocol did. On chains, giving a CLI agent a command sheet instead of help lowered its cost relative to the MCP server from 1.6–2.0 to 0.9–1.2, a wider swing than the gap between the best CLI configuration and the server. Our thesis follows: the case for CLI-first rests on the agents that build and check a product, and for agents in general, what they are told about an interface matters more than which protocol carries it. We report five findings:

- **F1.** An agent that learned the CLI from its help cost 1.6 to 2 times as much as an agent given the curated MCP server's schemas (Section 6.5).
- **F2.** For the same operations, an MCP server generated from the CLI cost 1.3 to 1.8 times as much as the curated one; it also returned larger results, so the effect of catalogue size alone is not isolated (Section 6.6).
- **F3.** On chains of dependent steps, the CLI with a 1.1k-token command sheet and composition could not be told apart from the curated server for the stronger model, Sonnet, and cost 1.2 times as much for the weaker, Haiku (Section 6.7).
- **F4.** On two tasks that repeat an operation, the same configuration cost 36–39% less than the curated server for both models; Haiku saved only on bulk entry, and where one write was needed there was no consistent advantage (Section 6.8).
- **F5.** Composition moved the unit a person could review from single actions to whole scripts, and agents on both protocols, in four of eight configurations, reported success they had not achieved (Section 6.9).

The paper contributes (1) a definition of CLI-first development (Section 3); (2) a single-product case study of the rule in practice (Section 4); (3) a caching-aware cost model that separates what an interface costs an agent into description, discovery and composition (Section 5); (4) an exploratory experiment with all runs, transcripts and analysis released (Section 6); and (5) principles for agent-ready CLIs and controls for agents that act through them (Sections 7–8).

2 RELATED WORK

Delegation and oversight. The debate between direct manipulation and delegation to agents is older than large language models [31, 55]. Norman warned that delegation works only when people keep an accurate sense of what the agent is doing and can take control back [44], and Horvitz's mixed-initiative principles [21] and Amershi et al.'s guidelines for human-AI interaction [2] turn that concern into design advice. Work on automation describes how oversight degrades: the operator of a highly automated system is left with the hardest decisions and the least practice at them [8], the right level of automation differs by stage of a task [46], and trust has to be calibrated to reliability [28]. An agent narrows the person's gulf of execution and can widen their gulf of evaluation [24], and stating intent is itself hard for non-experts [42, 63]. F5 bears on both gulfs.

Agent-computer interfaces and tool use. SWE-agent showed that interface design changes agent performance: a purpose-built interface resolved 18.0% of SWE-bench Lite tasks [26] against 11.0% for a plain shell [61]. CodeAct found that acting through executable code outperformed JSON and text tool calls by up to 20% in success rate across 17 models [56], and InterCode and Terminal-Bench evaluate agents in shells [35, 62]. Documentation alone lets models use unfamiliar tools about as well as demonstrations do [23], and ReWOO and LLMCompiler cut model invocations by planning tool calls ahead [27, 60]. Translating language into shell commands is a long-studied problem [29].

GUI, API, CLI and MCP agents. GUI agents beat CLI agents on 440 desktop tasks until the CLI's missing capabilities were filled in [67], and take 2.7 to 4.3 times more steps than necessary [1]; API-first and hybrid agents are faster [30, 65, 66], and MCP tools raise computer-use agents' success [25]. Comparisons of MCP and CLIs disagree. Across seven scaffoldings, paired MCP-to-CLI cost ratios ranged from 0.43 to 29 [18]; in 120 runs MCP was 23% faster and 2.5% cheaper than an equivalent CLI [64]. Holding a backend fixed, Smithery's 756 runs against public APIs found native MCP ahead of a CLI with descriptions on GitHub's REST API in success (91.7% against 83.3%), median tokens (28.8k against 82.9k) and time (10.4 against 24.9 s) [32, 43], and Scale Labs' 50 long-horizon tasks found the CLI's advantage depended on model and task and mattered little for frontier models [22].

Practice and lineage. "Agent experience" has become a design concern [7, 40], and products increasingly give the UI and the agent one set of actions [10, 49]. For CLI input, named flags beat single JSON payloads on accuracy and cost [33, 48], and Mistral gives every interactive prompt a flag [37]. The architecture underneath CLI-first is old: composable programs [34], ports and adapters [13], command-line design guidance [50] and API usability research [39]; the decay of a shared interface is a form of architecture erosion [14, 47]. CLI-first differs from API-first in naming which client must work first and giving it the role of reference.

How this study differs. The two fixed-backend studies are closest to ours. We differ in three ways: (1) we vary what the agent knows about the interface and whether it may compose as separate factors, so their effects can be told apart; (2) we decompose cost by token type under prompt caching and model it, rather than reporting totals; and (3) we score a production product's database state rather than an agent's answer.

3 CLI-FIRST DEVELOPMENT

A team practises CLI-first development when it holds two rules. First, a capability is complete only when it can be exercised from the product's CLI; other clients may ship it at the same time or later, but not instead. Second, the CLI is the reference client: when the web application, the MCP server or an embedded assistant disagrees with the CLI about what the product does, the disagreement is a defect in one of them, and checks written against the CLI define the expected behaviour. The first rule without the second produces coverage that can drift; the second without the first produces a reference for part of the product.

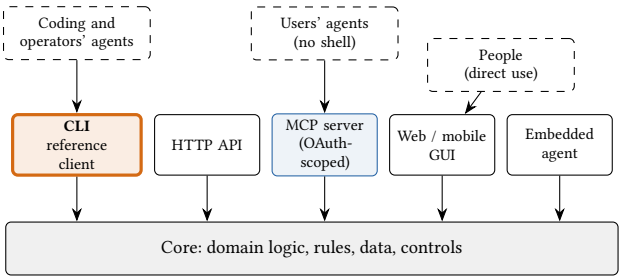


Figure 2: Every client calls one core; the CLI is the reference. Coding and operators' agents use the CLI; users' agents usually arrive through a scoped MCP server.

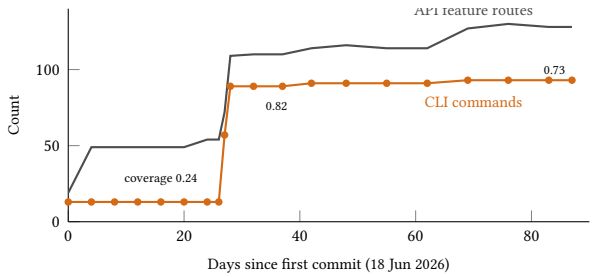


Figure 3: CLI coverage fell under a written rule until a dedicated phase restored it. Commands and API feature routes in Salli over its first 87 days; authentication routes are excluded.

Three kinds of agent interact with such a product (Figure 2). Coding agents and operators' agents run in a shell and are the CLI's main users: they exercise features without a browser, configure deployments and check their own changes. Embedded agents call the same core operations and get no more power than any other client. Users' agents, the assistants that end users delegate to, usually run where no shell exists and reach Salli through an MCP server with OAuth-scoped access. CLI-first serves them indirectly, by making every capability a scriptable operation that any protocol can expose; it does not guarantee that they will: Salli's open-source core has 125 CLI commands but 24 MCP tools.

The analogy with mobile-first [58, 59] holds for the order of work and fails on constraint: mobile-first started from the most limited client, CLI-first from the most capable, and its new user is an agent acting for a person. Accessibility is a closer precedent: the interfaces that expose an application to a screen reader stay complete only where teams treat gaps as defects.

4 A CLI-FIRST PRODUCT

Method. We studied Salli, a personal finance and tax product for Sri Lanka whose agent guidance adopts the CLI-first rule and which the author develops. The design is a single-case study with the author as participant-observer, in the tradition of autobiographical design [41] and following case-study guidance from software engineering [52]. The data are the product's git history (417 commits), architecture documents and commit messages at a fixed snapshot (13 September 2026). For CLI coverage we counted CLI commands and API route handlers, excluding authentication, at a series of dates; the ratio of counts is a proxy for parity, not a mapping of commands to routes. For agent involvement we counted commits whose message contains a `Co-Authored-By: Claude` trailer. The case can show what happened in one product; it cannot show how often CLI-first works elsewhere. Commit messages were themselves written with agent assistance and may be more explicit, and more favourable, than an independent record [9].

4.1 Coverage fell behind under a written rule and recovered in a dedicated phase

Salli combines a double-entry ledger, a versioned tax engine and AI agents that explain a user's financial position. Its domain core has no input/output or framework imports, and its CLI and HTTP API build their services through one composition function. The project's agent guidance states the order of work: "Core + CLI first — all domain logic runs from the terminal before any HTTP server exists," then the API, then web and mobile clients. 311 of its 417 commits carry an agent trailer.

The rule did not hold on its own. An HTTP API with 19 feature routes existed from the first day, and over four weeks the API grew while the CLI did not: by 14 July the CLI had 13 commands against 54 feature routes, a ratio of 0.24 (Figure 3). The roadmap then inserted a "CLI completion" phase ahead of new domain work, and within two days the ratio reached 0.82. Five domains built immediately afterwards each received core logic, API routes and a CLI command group within about twelve minutes, in one agent session. Coverage then fell to 0.73 as the API gained routes the CLI did not; the open-source core now runs a continuous-integration check that fails when a route has neither a command nor a recorded reason for lacking one. We have no data yet on whether the check holds coverage.

4.2 A pass through the CLI found a control that held on only one path

Once the CLI covered the product, a coding agent's pass through it found defects in the core. One agent-assisted commit (`862ed06`, 19 July) records the defects "found during a full CLI end-to-end pass". The CLI had never set up the checkpointer that the HTTP server configures for the embedded agents, so the agents' history commands crashed and the human-approval gate on their write tools silently did nothing when invoked from the CLI. Data export crashed for any user who had computed their tax, and a command that posts imported transactions misreported how many it had posted. The approval-gate defect could not be seen from the web client, which reached the agents through the HTTP server that set the checkpointer up. That gate had been fixed for the web path a month earlier; the CLI pass found that the same guarantee did not hold on another path into the core. These observations, from one product and one participant-observer, motivate study S2 in Section 10.

Table 1: A pipeline needs two passes where direct calls need k+1. Input-token equivalents for $k = 5$ dependent operations with $S = 10$k, $u = 1$k, $a_i = 100$, $r_i = 2$k, $c = 400$, $f = 200$, $\omega = 1.25$, $\rho = 0.1$. The last row drops tool definitions from the prefix. The table uses the five-minute cache rate for illustration; the experiment used one-hour entries ($\omega = 2$).

	Passes	Uncached	Cached
Direct tool calls	6	97.5k	34.5k
CLI pipeline	2	22.6k	15.6k
CLI pipeline, $S = 1$k	2	4.6k	3.5k

5 WHAT AN INTERFACE COSTS AN AGENT

An MCP server publishes typed tools whose definitions the agent's host loads into the model's context [38]; the model calls tools one response at a time, possibly several in parallel. With a CLI that emits structured output, an agent can instead write a chain of operations once as a program and run it as one tool call, so that intermediate data move through pipes or files rather than through the model, as in Figure 1.

A cost model. Consider k operations, each needing the result of the one before. Let S be the fixed prompt prefix (system prompt and interface description), u the user's request, a_i and r_i the tokens of the i-th call and its result, c the tokens of a pipeline script and f its filtered output. With direct calls there are $k + 1$ model passes, and pass j reads $P_j = S + u + \sum_{i<j}(a_i + r_i)$. Without caching, the input tokens processed are

$$I_{\text{calls}} = (k+1)(S+u) + \sum_{i=1}^{k}(k+1-i)(a_i + r_i), \tag{1}$$

quadratic in k when results are of similar size, while a single pipeline needs two passes:

$$I_{\text{pipe}} = 2(S+u) + c + f. \tag{2}$$

With prompt caching, cache writes cost ω times the base input rate and cache reads ρ times; for one provider $\rho = 0.1$, ω is 1.25 for five-minute and 2 for one-hour entries, and output costs five times the input rate [6]. Direct calls then cost approximately $\omega P_1 + \rho \sum_{j \le k} P_j + \omega \sum_i (a_i + r_i)$ input-token equivalents and the pipeline $\omega(S + u + c + f) + \rho(S + u)$ (Table 1). Caching removes most of the quadratic term but not the $k + 1$ sequential passes, nor the cost of every intermediate result entering the context once. When many agents share a prefix, even the first pass may read S from the cache.

Discovery and repetition. Two further terms decide whether a pipeline is available. The first is *discovery*. Equation 2 assumes an agent that knows the commands and the shape of their output. An MCP server supplies that knowledge in S; an agent that must learn a CLI reads help instead, taking d extra passes that add h tokens, written to the cache once and re-read on every later pass. Whether this exceeds the CLI's saving in S depends on d, h and the passes that follow. The second is *repetition*. For n independent operations of one kind, direct calls can run in parallel within one pass, but the model writes all n calls and all n results enter the context. A loop over a CLI command avoids the per-call overhead; its length is constant in n when it iterates over the product's own data, as in Figure 1, and grows with n when the arguments must be written out.

Research questions. The model raises four questions. We formed the predictions after the first wave of the experiment (Section 6.2) and test them on data that include that wave, so they are exploratory.

RQ1, knowledge. Does an agent that must learn a CLI from its help cost more than one given an MCP server's schemas? *Predicted: yes.*

RQ2, description size. For the same operations, does an MCP server generated from the CLI cost more than a curated one? *Predicted: yes.*

RQ3, composition. When the agent knows the commands, does a composed CLI cost less than the curated MCP server on chains of dependent operations, and on tasks that repeat an operation? *Predicted: yes, more so for repetition.*

RQ4, oversight. What does composition do to what a person can review? *No prediction.*

Composition's economics come from code as the agent's action space, not from the shell; MCP tools can also be called from code in a sandbox [3, 4, 11], though standard MCP clients do not offer it. A CLI brings composition with the interface, along with costs: scripts must anticipate output shapes that MCP declares, chains can fail halfway, shell quoting is fragile [33], every invocation starts a process, and a shell carries ambient authority. A pipeline planned before reading data keeps untrusted content out of the control flow [15]; one that interpolates data into commands, like Figure 1's loop, does not.

6 AN EXPLORATORY EXPERIMENT

6.1 Design

We held one product, its data and its tasks fixed and varied the interface. The product was Salli's open-source core at commit `0277747`, whose CLI (125 commands, 120 with `--json`) and MCP server (24 tools) call the same services and database. A *configuration* fixes three factors (Table 2). The *protocol* is MCP tool calls or shell commands. *Knowledge* is what the interface description tells the agent before it starts. MCP configurations receive tool schemas. CLI configurations receive one of four descriptions: help alone; the core's agent skill, pasted into the prompt (646 tokens naming command groups and rules, but no options); the core's five skills, installed for the agent to load if it chooses; or a *command sheet* generated from the CLI's own definitions (1,134 tokens giving usage, options and JSON output shape for the 14 commands that do what MCP-24's tools do). *Composition* is whether the agent may combine commands: one-per-call configurations permitted only the `salli` executable, while composed ones asked the agent to compose dependent steps and allowed pipes, `jq`, scripts and (from wave 4) Python. MCP-24 is the core's own, curated server, whose results are compact summaries. MCP-102 is generated from the CLI, with one tool for each of 102 of the 120 commands that support `--json` (administrative command groups are left out), and returns the CLI's JSON, so it differs from MCP-24 in both catalogue size and result size.

Table 2: Eight configurations cross protocol, knowledge and composition. Knowledge gives the interface description's size (`o200k_base` tokens). 1st pass: median context of the first model call. Runs: retained runs (Haiku and Sonnet).

Configuration	Knowledge	Composition	1st pass	Runs
MCP-24	24 schemas (3.0k)	parallel calls	5.3k	96
MCP-102	102 schemas (9.7k)	parallel calls	17.2k	96
CLI skill/step	skill (0.6k)	one per call	3.8k	64
CLI skill/comp	skill (0.6k)	composed	3.9k	96
CLI help/comp	help only	composed	2.9k	64
CLI installed/comp	5 skills, on demand	composed	6.6k	96
CLI sheet/step	command sheet (1.1k)	one per call	4.7k	96
CLI sheet/comp	command sheet (1.1k)	composed	4.7k	96

What the experiment can and cannot show. It compares these configurations on one product, six tasks and two models from one provider. It cannot say how often CLIs beat MCP servers in general, and it does not test the process claim of Section 4.

Tasks and scoring. Six tasks could all be completed with the operations available to MCP-24. Four are *chains* of $k = 1, 2, 4$ and 8 operations, with nominal dependency depths 1, 2, 4 and 6: report the tax owed; record a cash expense; record a salary, compute the tax and set a reminder carrying the figure; and create an account, post to it, find the expense account with the largest balance, refund it, and set a reminder carrying the tax and the bank balance. Two are *repetition tasks*: record 25 expenses listed in the request (bulk entry), and set a reminder for every expense account over LKR 20,000 (fan-out, Figure 1). Each ran on a small ledger (16 accounts, 30 entries) and a large one (106 accounts, 440 entries); fan-out needs one write on the small ledger and 24 on the large, which makes the small cell a control where composition has nothing to compose. A *cell* is one task on one ledger, so there are 12 cells and 24 combinations of model and cell. Every run used a freshly seeded user. The one-operation tax question was scored from the answer and the other tasks against the database, by the expected records; the two-step and repetition tasks also required that no other records of the same kind exist on the task's dates, while the 4- and 8-step checks did not look for extra records.

Interface asymmetries. MCP-24's reminder tool takes free text, while the CLI's `reminders add` takes it as a `KIND` argument that neither the skill nor the sheet explains. The CLI accepts account codes. The core's documentation, our prompt for the CLI configurations and the command sheet's header all said identifiers could be shortened to a unique prefix, which `entry add` did not accept. On the small ledger the 8-step task's target account is necessarily the one created in its first step.

Harness. The agent was Claude Code 2.1.293 in headless mode driving Claude Haiku 5.5 and Claude Sonnet 5.5. Its system prompt was replaced by a short task prompt common to all configurations plus a configuration-specific paragraph; user settings were excluded, MCP tool search disabled, and only the configuration's tools enabled. The composed configurations' paragraph alone gave a reason to compose, so "that intermediate output does not have to pass through you"; no other configuration was told anything about efficiency. MCP-24 ran the core's server over stdio with its OAuth user lookup replaced by the seeded user. Composed configurations ran with permissions bypassed under a deny-list of deletion, network, version-control and package-manager commands, and with a directory holding only `salli` prepended to the path; there was no sandbox, and the environment held the test instance's database URL and service keys. No transcript shows a command that bypassed the CLI. Caching used one-hour entries shared across runs: in 702 of 704 runs the first call read the prompt and interface description from a cache warmed by earlier runs.

Table 3: The design grew over five waves. Parallel: concurrent runs. Retained: runs in the analysis.

Wave	Content	Parallel	Retained
1	Pilot: MCP-24, CLI skill/step, skill/comp, help/comp; chains	6	128
2	Command-sheet configurations; composed configurations re-collected	8	224
3	CLI installed/comp	8	55
4	MCP-102 on chains; repetition tasks for six configurations	8	217
5	Shared-directory runs, re-collected one at a time	1	80

6.2 Collection history

The design grew in five waves, each started after the previous one's results were visible (Table 3). The first wave, a pilot on the chain tasks, found the CLI more expensive than MCP-24; the command-sheet configurations, the repetition tasks and the research questions came afterwards, and the pilot's 128 MCP-24 and CLI skill/step runs are part of the analysed data. Of 947 runs attempted, 243 were superseded and 704 retained; the run ledger lists each with its wave, parallelism, permissions and fate. Four harness problems caused the supersessions. Claude Code's allowlist matcher refused composed commands containing variables, quoting or loops (in 29 of 128 wave-1 composed runs, all of which still succeeded), so composed configurations were re-collected without it (128 runs). A deny-list that also blocked interpreters was in force for 32 repetition-task runs, three of which had a script refused; all 32 were re-collected. The composed chain runs we kept used that deny-list, and none invoked an interpreter. Parallel runs shared a temporary directory and overwrote each other's files, so the 80 composed runs that had used a shared path, and one that ran beside a smoke test, were collected again one at a time; because this rule selected runs by behaviour, Section 6.10 restores them. One run was stopped by the operator in error and one produced no output; both were collected again.

6.3 Measures and analysis

For each run we recorded success; cost; tokens of each kind, summed over the run's agent invocations; passes, tool calls and characters of tool output; and time as reported by the agent harness, summed over invocations. Cost is at list prices: \$0.10, 0.20, 0.01 and 0.50 per million uncached input, cache-write, cache-read and output tokens for Haiku, and \$2, 4, 0.20 and 10 for Sonnet. These prices reproduce the harness's reported cost for every run. The outcome is cost relative to MCP-24: in each

cell the ratio of geometric-mean costs, combined over cells by the geometric mean. We report two targets of inference. *For these tasks*, the 95% interval is a Welch-type t interval for the mean log ratio over cells, with runs within cells as the sampling unit and Satterthwaite degrees of freedom; p-values come from a stratified permutation test (10,000 permutations), Holm-adjusted over 24 comparisons [20]. A bootstrap over runs within cells [17] gives similar intervals and is in the released data. *For tasks like these*, a t interval over tasks uses each task's log ratio averaged over its two ledgers; with four chain tasks it is wide, and with two repetition tasks uninformative. We compare models as a ratio of ratios, overall and per task, and give absolute differences in cents.

6.4 Results

Success was at ceiling, so cost decides. Of 704 runs, 693 (98.4%) succeeded; per configuration and model, success ranged from 57/60 to 60/60 with overlapping Wilson intervals. Figure 4 and Table 4 give every cost effect; Table 5 gives the median cost per run that anchors them, from 0.05 cents (Haiku, MCP-24, one operation) to 70.9 cents (Sonnet, CLI sheet/step, bulk entry on the large ledger).

6.5 F1. Agents that learned the CLI from its help paid 1.6 to 2 times as much

An agent that had to learn the CLI from its help cost more than an agent given MCP-24's schemas (RQ1). On chains CLI help/comp cost 1.99 times as much with Haiku and 1.62 times with Sonnet, about 0.2 and 1.7 cents more per task, with intervals above 1 for these tasks and for tasks like these; it cost more in all 8 cells for Haiku and 7 of 8 for Sonnet. These agents made two to four help calls per run, and at $k = 8$ took a median of 15.5 passes against 7 for MCP-24 (Haiku). A skill that named the command groups but no options did not close the gap (1.22 to 2.08).

6.6 F2. The generated MCP server cost more than the curated one in every cell

MCP-102 cost more than MCP-24 in all 12 cells for both models (RQ2): 1.79 and 1.57 times as much on chains with Haiku and Sonnet, and 1.53 and 1.34 times on repetition tasks ($p = 0.002$ throughout). Its schemas took 9.7k tokens against 3.0k, and it also returned 2.3 to 3.4 times as many characters of results, so the comparison does not isolate catalogue size. Against the CLI it was generated from, used with the command sheet and composed, it was the more expensive interface on both models and both task families.

6.7 F3. On chains, the CLI with the command sheet could not be told apart from MCP for Sonnet, and cost more for Haiku

With the command sheet and composition, Sonnet's agents cost about what MCP-24's did on chains, and Haiku's cost more (RQ3). Sonnet's ratio was 0.89 (95% CI 0.78–1.02; Holm $p = 0.26$), under a cent less per task; over tasks like these the interval, 0.57–1.39, is too wide to claim either parity or a saving. Haiku's ratio was 1.21 (1.11–1.31; Holm $p = 0.002$), about 0.07 cents more per task. The models differed: Sonnet's ratio was 0.74 times Haiku's (0.67–0.82). At $k = 8$ on the large ledger Sonnet composed its work into three tool calls and four passes, and Haiku composed its work into ten calls and nine passes.

Result size is a third factor. The CLI's JSON is fuller than MCP-24's summaries. With one command per call, CLI sheet/step agents received about twice as many characters of results as MCP-24's (1.98 times for Haiku, 1.96 for Sonnet) and cost 1.28 and 1.24 times as much, although, like MCP-24's agents, they issued one operation per call. Composed agents could filter results before they reached the model, and received 1.64 and 1.07 times as many characters.

6.8 F4. Composition paid when an operation repeated

On the two repetition tasks, the composed CLI with the command sheet cost 36–39% less than MCP-24 (RQ3): 0.64 with Haiku (95% CI 0.56–0.73; Holm $p = 0.005$), about 0.3 cents less per task, and 0.61 with Sonnet (0.50–0.74; $p = 0.002$), about 4 cents less. Cost fell in three of four cells for Haiku and four of four for Sonnet. **The saving was not spread alike.** Haiku saved on bulk entry only (0.41; fan-out 1.02), while Sonnet saved on both (0.67 and 0.56). Sonnet's ratio was 1.65 times Haiku's on bulk entry and 0.55 times on fan-out, so the overall ratio of ratios, 0.95, hides an interaction between model and task. In the control cell, the fan-out that needs one write, composition had nothing to compose and gave no consistent advantage: MCP-24 was cheaper for Haiku (0.11 against 0.18 cents) and more expensive for Sonnet (1.83 against 1.61). Two tasks cannot establish generality; the interval over tasks spans 0.20 to 1.91 for Sonnet.

Composition without the sheet had similar point estimates that were not established (CLI skill/comp, 0.79 for Haiku and 0.64 for Sonnet; Holm $p = 0.19$ and 0.067), and restoring the superseded runs moves Sonnet's to 0.81. One command per call reversed the effect for Sonnet, at 2.90 times MCP-24. On bulk entry its agents issued the 25 writes in parallel, with account identifiers shortened to a prefix as the documentation, our prompt and the command sheet said they could be; `entry add` rejected each with a traceback, the agents repeated the writes, and the two bulk cells cost 5.7 and 6.6 times MCP-24.

6.9 F5. Composition coarsened oversight, and neither protocol made reports reliable

The configuration that saved most put 25 writes behind one tool call (RQ4). With MCP-24 or one command per call, each tool call carried at most one write; on the bulk task the composed agents placed a median of 25 writes in a single call. No person approved anything in the experiment, so this describes what a reviewer would be shown, not how reviewers would fare: per-action approval would become approval of a script. In 31 of the 96 composed repetition runs the script also sent the writes' output to `/dev/null`, so not even the transcript records what each write returned. **Five of the 11 failures ended in a report of success**, 5 of 704 runs, as coded by the author. Two were with MCP-24 (an omitted account; the wrong one of two similar names), one each with MCP-102 and CLI sheet/step (each refunded the wrong

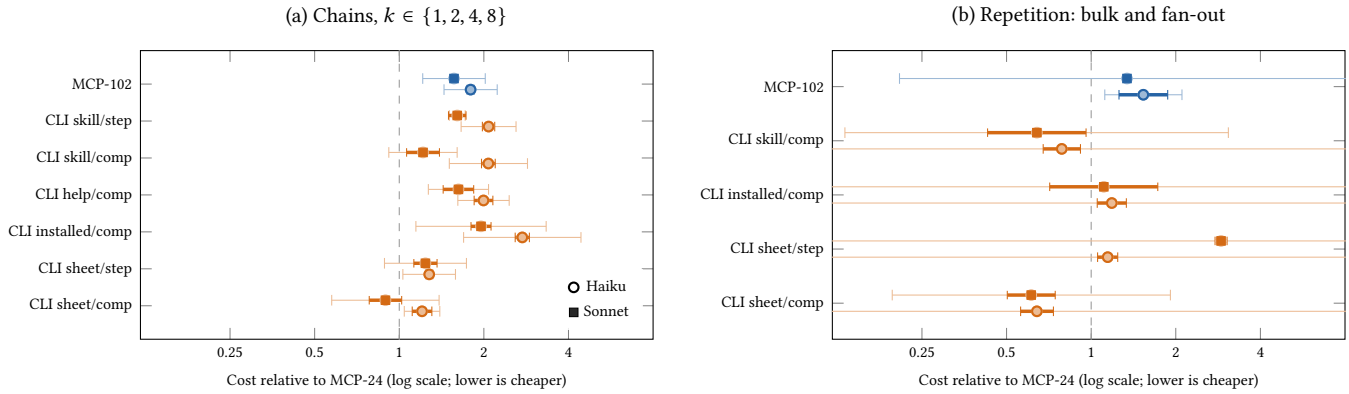


Figure 4: Knowledge, more than protocol, decides cost; composition pays when work repeats. Cost of each configuration relative to MCP-24 (dashed line), geometric mean over cells; blue: MCP, orange: CLI; hollow: Haiku, filled: Sonnet. Thick bars: 95% Welch intervals for these tasks. Thin bars: 95% intervals over tasks, clipped to the axis for the two repetition tasks. CLI help/comp and CLI skill/step were not run on repetition tasks.

Table 4: Without the command sheet, every CLI configuration cost more than MCP-24 on chains; with the sheet and composition, the CLI cost less on repetition tasks. Ratio: geometric mean over cells (lower is cheaper). For these tasks: 95% Welch t interval, runs within cells. For tasks like these: 95% t interval over tasks. p: stratified permutation test, Holm-adjusted over the 24 comparisons in this table; the smallest attainable value is 0.002.

	Haiku 5.5				Sonnet 5.5			
Configuration	ratio	these tasks	tasks like these	p	ratio	these tasks	tasks like these	p
Chains (4 tasks, 8 cells)								
MCP-102	1.79	1.74–1.85	1.44–2.23	0.002	1.57	1.53–1.60	1.21–2.02	0.002
CLI skill/step	2.08	1.98–2.19	1.66–2.60	0.002	1.61	1.50–1.73	1.51–1.72	0.002
CLI skill/comp	2.08	1.96–2.19	1.51–2.86	0.002	1.22	1.06–1.39	0.92–1.61	0.061
CLI help/comp	1.99	1.85–2.15	1.62–2.46	0.002	1.62	1.43–1.84	1.27–2.08	0.002
CLI installed/comp	2.74	2.58–2.90	1.69–4.43	0.002	1.95	1.80–2.12	1.15–3.33	0.002
CLI sheet/step	1.28	1.24–1.32	1.03–1.58	0.002	1.24	1.13–1.36	0.89–1.73	0.002
CLI sheet/comp	1.21	1.11–1.31	1.04–1.39	0.002	0.89	0.78–1.02	0.57–1.39	0.257
Repetition (2 tasks, 4 cells)								
MCP-102	1.53	1.26–1.87	1.12–2.10	0.002	1.34	1.31–1.37	0.21–8.62	0.002
CLI skill/comp	0.79	0.68–0.92	<0.1–>10	0.186	0.64	0.43–0.96	0.13–3.08	0.067
CLI installed/comp	1.18	1.05–1.33	<0.1–>10	0.282	1.11	0.71–1.73	<0.1–>10	0.605
CLI sheet/step	1.15	1.05–1.24	<0.1–>10	0.061	2.90	2.76–3.04	<0.1–>10	0.002
CLI sheet/comp	0.64	0.56–0.73	<0.1–>10	0.005	0.61	0.50–0.74	0.20–1.91	0.002

account), and one with CLI skill/comp (reminder texts carrying stray quotation marks). One MCP-24 agent ended: "Created 23 reminders … for the expense accounts over LKR 20,000", when 24 qualified. The other six failures disclosed what had gone wrong.

6.10 Composition saved tokens rather than passes, and the conclusions survive a cold cache

Composition saved tokens rather than passes. Across the 24 combinations of model and cell, CLI sheet/comp took more passes than MCP-24 in 8, fewer in 4 and as many in 12; its saving came from what each pass carried. Table 6 decomposes two Sonnet cells. On the 8-step chain, CLI sheet/comp read 36k cached tokens against 64k for MCP-24, and three quarters of its saving came from smaller cache writes; the help-only agent took twice the passes and read twice as much. MCP-24 took 5 passes rather than $k + 1 = 9$ because it issued independent calls in parallel. On the fan-out, smaller cache writes (54%) and fewer output tokens (44%) made up the saving; on Haiku's bulk task output tokens made up 63%.

On chains, the command sheet mattered more than composition, and installed skills cost most. Holding composition fixed, replacing help with the sheet lowered the chain ratio from 1.99 to 1.21 (Haiku) and 1.62 to 0.89 (Sonnet); holding the sheet fixed, composition lowered it from 1.28 to 1.21 and 1.24 to 0.89. On repetition tasks composition did more, lowering the ratio from 1.15 to 0.64 and 2.90 to 0.61. Which part of the sheet mattered (scope, options or output shapes) is not identified. Installed skills were loaded

Table 5: Absolute cost anchors the ratios. Median cost per run in US cents at list prices (lower is cheaper), with each configuration's ratio to MCP-24. (a) Chains by number of operations k, pooled over ledgers. (b) Repetition tasks by ledger; Fan S is the one-write control.

(a) Chains	Haiku 5.5					Sonnet 5.5				
Configuration	k=1	2	4	8	ratio	k=1	2	4	8	ratio
MCP-24	0.05	0.15	0.24	0.44	1.00	1.15	2.89	5.21	7.03	1.00
MCP-102	0.12	0.29	0.40	0.71	1.79	1.52	5.51	7.28	11.00	1.57
CLI skill/step	0.13	0.29	0.57	0.88	2.08	1.73	5.09	8.77	11.33	1.61
CLI skill/comp	0.15	0.33	0.41	0.74	2.08	1.82	2.34	7.49	6.46	1.22
CLI help/comp	0.13	0.25	0.42	0.79	1.99	2.03	5.57	8.07	9.30	1.62
CLI installed/comp	0.21	0.36	0.63	0.94	2.74	2.96	7.85	7.79	8.74	1.95
CLI sheet/step	0.06	0.26	0.34	0.58	1.28	1.07	4.87	7.42	9.46	1.24
CLI sheet/comp	0.06	0.15	0.23	0.59	1.21	1.07	3.62	3.49	4.99	0.89
(b) Repetition	Bulk S	Bulk L	Fan S	Fan L	ratio	Bulk S	Bulk L	Fan S	Fan L	ratio
MCP-24	0.60	0.76	0.11	0.71	1.00	9.91	12.52	1.83	11.70	1.00
MCP-102	1.51	0.90	0.20	0.94	1.53	10.46	16.05	3.10	16.95	1.34
CLI skill/comp	0.32	0.36	0.22	0.53	0.79	8.11	4.20	2.60	5.10	0.64
CLI installed/comp	0.41	0.72	0.38	0.73	1.18	5.77	9.38	5.49	8.40	1.11
CLI sheet/step	0.56	0.86	0.16	0.99	1.15	64.90	70.91	2.46	15.86	2.90
CLI sheet/comp	0.23	0.30	0.18	0.47	0.64	6.91	8.64	1.61	4.40	0.61

Table 6: The composed agent wrote less and re-read less. Median tokens (thousands), passes, tool calls and cost (cents) in two Sonnet cells on the large ledger.

Config.	Cache write	Cache read	Out	Passes	Calls	Cost
Sonnet, $k = 8$ chain, large ledger						
MCP-24	16.8	63.9	1.3	5	8	9.3
MCP-102	27.7	193.5	1.4	6	9	16.3
CLI help/comp	15.9	128.4	1.7	11	10	11.0
CLI skill/comp	11.1	108.9	1.9	11	10	8.8
CLI sheet/comp	10.9	36.4	1.0	4	3	6.1
Sonnet, fan-out to 24, large ledger						
MCP-24	17.6	25.2	4.1	3	26	11.7
MCP-102	27.3	70.1	4.8	3	26	16.9
CLI skill/comp	8.0	32.9	1.1	5	4	5.1
CLI sheet/comp	7.7	17.2	0.9	3	2	4.4

in 35 of 60 Haiku and 30 of 36 Sonnet runs, arrived as an extra pass, and made CLI installed/comp the most expensive configuration on chains (2.74 and 1.95).

The conclusions survive a cold cache and the superseded runs. Re-pricing every run as if its first pass had written the cache raises MCP-102's chain ratio to 2.44 and 2.38 and lowers CLI help/comp's to 1.41 and 1.17. CLI sheet/comp's ratios move by at most 0.11 and CLI sheet/step's by up to 0.32; two ratios that were near 1 and not significant move just below it (CLI skill/comp, Sonnet, chains, 1.22 to 0.99; CLI installed/comp, Sonnet, repetition, 1.11 to 0.97). Restoring the superseded shared-directory runs changes no ratio by more than 0.08 except CLI skill/comp on Sonnet's repetition tasks, from 0.64 to 0.81.

The curated MCP server was fastest, and the CLI failed in ways MCP-24 did not. In time reported by the agent harness, MCP-24 took a median of 9.5 seconds per run on chains (8.6 for Haiku, 11.5 for Sonnet), against 15.3 to 39.3 for the CLI configurations and 16.3 for MCP-102, which like the CLI starts a Python process per call. On repetition tasks it took 26.5 seconds, against 60.3 to 69.2 for every other configuration. Runs executed six to eight at a time in waves 1–4 and one at a time in wave 5. Raw Python tracebacks of up to 9,400 characters entered the context in 48 of 512 CLI runs and 5 MCP-102 runs; in 35 of these 53 runs they came from identifiers shortened to a prefix, which `entry add` rejected although the documentation and our command sheet said it would accept them. Outputs over the harness's 30 KB limit were truncated to a preview in 56 CLI runs. Five composed runs took 11.6 to 15.9 minutes: in two, a command the agent wrote waited on standard input until the harness moved it to the background; in two, the agent called the trial balance once per account; one we could not explain. MCP-24 met none of these problems.

Table 7: Each finding, its evidence, and how far it reaches.

	Finding and evidence	Scope
F1	A CLI learned from help cost 1.62–1.99× MCP-24 on chains	Both models; holds over tasks
F2	MCP-102 cost more than MCP-24 in all 12 cells, both models	Confounded with result size
F3	Sheet + composition on chains: 1.21× (Haiku), 0.89× (Sonnet, 0.78–1.02)	Sonnet not distinguishable from MCP-24
F4	Sheet + composition on repetition: 0.64× (Haiku), 0.61× (Sonnet)	Two tasks; Haiku on bulk only
F5	25 writes per tool call; 5 of 11 failures reported as successes	Both protocols; no human reviewers

6.11 Threats to validity

Construct. Cost was measured at one provider's prices under a warm shared cache; the cold-start analysis bounds that effect. Scoring checked the expected records, and same-kind extras on three

of the six tasks, not the whole state; every failure was also read by hand. The command sheet was generated for the operations the tasks needed and is a best case for CLI knowledge, so we also report help-only and skill configurations. **Internal.** The design changed between waves, and three permission regimes and two degrees of parallelism are confounded with configuration and family; the run ledger records each run's conditions, and restoring the superseded runs changes one ratio materially. Only the composed configurations were given a reason to compose, and only the CLI configurations were told about prefix identifiers. **External.** One product, one harness, two models from one provider, and tasks drawn from what MCP-24 can do. Salli's CLI is absent from training data, unlike `git` or gh; CLI help/comp approximates an unfamiliar CLI, and a familiar one is left to the confirmatory study (Section 10). **Statistical conclusion.** With three to five runs per cell and two to four tasks per family, we report intervals for these tasks and for tasks like these separately, correct for 24 comparisons, and treat the research questions as exploratory. The author built the product, tasks and harness (see the positionality statement).

7 PRINCIPLES FOR AGENT-READY CLIS

Table 8 lists nine principles. Most restate good CLI design [50] and advice on writing tools for agents [5]; the evidence column separates prior studies, the case, the experiment and practitioner reports.

8 CONTROLS FOR AGENTS THAT ACT THROUGH A CLI

An agent with a shell, a person's credentials and access to untrusted content is exposed to indirect prompt injection [16, 19]; with network access it has all three elements of Willison's lethal trifecta: private data, untrusted content and a way to send data out [57]. A coding agent that reads content generated from users' documents while holding an operator's credentials fits this description, and OWASP names the risk excessive agency [45]. Rules therefore have to be enforced in the core. We propose five controls and report which ones Salli has.

C1, approval out of band. A destructive or irreversible command should create a pending action on the server and return its identifier; the action runs only after a person approves it on a separate channel. A `--yes` flag the agent can pass is not consent. F5 makes this pressing: approval should attach to a script's effects, not its text. Salli implements it only for return preparation, and its approval gate for the embedded agents once failed open on the CLI path (Section 4).

C2, agent identities with least privilege. Agents should hold short-lived, scoped, read-only-by-default credentials distinct from the person's. Salli scopes external agents through OAuth for MCP, but at the studied commit its CLI does not authenticate at all, acting as whichever user its local configuration names. Credentials in environment variables or arguments are visible to child processes and transcripts.

C3, deterministic computation where correctness matters. Figures people act on should come from code, with the model explaining them, as in Salli's rule that the model never computes money or tax.

C4, attributable audit. Every action should be logged with the acting agent and the person it acted for. At the studied commit, Salli logs MCP tools' actions but not CLI commands', so the surface we recommend for agents is the one that is not audited.

C5, containment below the shell. Agents that run CLIs should do so in sandboxes with restricted file-system and network access, using signed, pinned releases. Command allowlists sit at the wrong level for composition: the harness's matcher refused composed calls in 29 of 128 runs, and our own harness, which then bypassed permissions under a deny-list without a sandbox, is an example of what teams do instead. Composed scripts add partial failure without rollback, retries without idempotency keys, and data interpolated into commands, so writes should be idempotent and every rule enforced in the core.

9 DISCUSSION

What CLI-first does and does not buy. The experiment does not support the claim that a CLI is the cheaper way for any agent to use a product (Table 7). A curated MCP server was cheaper than a CLI the agent had to learn, and faster. Its agents met no tracebacks, truncated outputs or stalled commands, and each of its actions could be reviewed on its own. Since users' agents reach Salli through MCP, nothing here argues for moving them. What the evidence supports is narrower: a CLI with a compact command sheet cost about as much as the curated server on chains for Sonnet, less on two repetition tasks, and less than a server generated from the same commands. The process claim, that a complete CLI lets coding agents build and verify the product, rests on the case and is the subject of study S2 in the agenda below.

Description is a design object. The largest effects came from what the agent was told, not from the protocol. On chains, the command sheet did more than composition; a skill that taught rules without options did little; and skills loaded on demand cost more than a description supplied up front. On repetition tasks, composition did most of the work. For MCP, the curated server's advantage over the generated one points the same way, though that comparison also varies result size. Both suggest treating the interface description as something to design and measure. The newest models may need less of it [22].

Delegation, oversight and access. Composition trades the granularity of oversight for cost, and on both protocols agents sometimes reported success they had not achieved. A CLI is not itself accessible to people [53], though people who delegate do not use it. Whether delegation through agents improves access, and how people review composed actions, is the subject of study S4.

Limits. We did not measure the cost of maintaining coverage. CLI checks cannot see layout, rendering or client-side state, and checks that put a language model in the loop are no more deterministic than the browser tests they replace.

Table 8: Nine principles for agent-ready CLIs, with the evidence for each: prior work (cited), the case (§4), the experiment (§6) or practitioner reports.

	Principle	Evidence and notes
P1	Cover the product, and check it	Gaps send agents back to the GUI [67]; coverage fell in Salli until enforced in continuous integration (Figure 3).
P2	Never wait for input	A flag for every interactive prompt [37]. The experiment's stalls came from commands the agents wrote, not from Salli's prompts, so the evidence is practitioners'. Consent is the exception (C1).
P3	Compact structured output	`--json` on commands that return data, accepted as input by related commands; every composed run depended on it. The CLI's JSON was about twice the size of MCP-24's summaries, and one-per-call agents paid for it (F3).
P4	Named flags for input	Flags beat single JSON payloads on accuracy and cost [33]; one study, five runs per model.
P5	A compact command sheet that matches behaviour	Documentation substitutes for demonstrations [23]; a 1.1k-token sheet of usage, options and output shapes cost less than help alone, a skill without options or skills loaded on demand (F1, F3). Which element mattered is not identified. A sheet that promises what the CLI does not do is costly: our sheet's claim about prefix identifiers led to most tracebacks.
P6	Errors that name the problem; never a traceback	Tracebacks of up to 9,400 characters entered the context in 48 of 512 CLI runs, 35 of 53 affected runs from identifiers that the documentation and command sheet said were accepted.
P7	Preview before change	`--dry-run` or diff for commands that change state [50]; a preview should return a plan identifier to which approval (C1) can be bound.
P8	Safe under concurrent use, with batch operations	Agents issued up to 25 invocations of one command in parallel; credential refresh, writes and connections must tolerate it. A command that takes many records in one call would give one-per-call agents part of composition's saving.
P9	Built on the shared contract, described sparingly	One composition function serves Salli's CLI and API; an interface generated from the CLI cost more as MCP than the curated one (F2).

10 RESEARCH AGENDA

Table 9 lists four studies; S1 and S3 extend RQ1–RQ3 of Section 5. For S1, a confirmatory version of the experiment should be preregistered with a task-level analysis and use about 20 chain and 30 to 50 repetition tasks generated from a specification, three repeats, at least three models from two providers and two harnesses, isolated caches, a frozen harness and randomised run order. It should include an MCP server whose tools match the command sheet one for one, MCP with code execution, a CLI agents already know (gh), tasks beyond the curated server's operations, and whole-state checks. Its cost in model calls is small; writing and verifying tasks is the constraint.

11 CONCLUSION

We have argued that a product's capabilities should first become usable from its command line, and that this CLI should be the reference client against which its other clients are checked. In one product, coverage fell behind until a dedicated phase restored it, and a pass through the CLI found an approval gate that worked on only one path into the core. Holding the product fixed, we found that a CLI is not cheaper for agents because it is a CLI. Cost depended mostly on what the agent knew. Composition paid when an operation repeated, and the curated MCP server stayed fastest and easier to review. For a CLI-first product, the description an agent receives, a command sheet for the CLI and a curated catalogue for MCP, is part of the interface and should be designed and measured like the rest of it.

Positionality and competing interests. The author founded Leaf Monkey Labs, which develops Salli and offers a hosted version commercially, and proposed the CLI-first thesis. The author built the product, the tasks, the command sheet and the experimental harness. The experiment's adverse findings for the CLI are reported in full.

Artifact availability. Salli's core is open source under the AGPL-3.0 at https://github.com/leafmonkeylabs/salli-core; the experiment used commit `0277747`. Its public history begins when the core was separated from the hosted product in October 2026, so Section 4 was measured on the earlier private history. The paper's source, the case-study scripts, the harness, the generated command sheet and MCP server, all 704 transcripts, the 243 superseded runs, the run ledger, the collection logs and the analysis are at https://github.com/leafmonkeylabs/cli-first-paper.

Use of generative AI. Generative AI tools were used for literature search, repository analysis, the experimental harness and drafting, and to simulate peer review of earlier drafts. The author checked every figure and quotation against its source and takes full responsibility for the content.

REFERENCES

[1] Reyna Abhyankar, Qi Qi, and Yiying Zhang. 2025. OSWorld-Human: Benchmarking the Efficiency of Computer-Use Agents. arXiv:2506.16042

[2] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*. ACM, 1–13. https://doi.org/10.1145/3290605.3300233

[3] Anthropic. 2025. Code Execution with MCP: Building More Efficient AI Agents. https://www.anthropic.com/engineering/code-execution-with-mcp

[4] Anthropic. 2025. Introducing Advanced Tool Use on the Claude Developer Platform. https://www.anthropic.com/engineering/advanced-tool-use

[5] Anthropic. 2025. Writing Effective Tools for AI Agents, Using AI Agents. https://www.anthropic.com/engineering/writing-tools-for-agents

[6] Anthropic. 2026. Prompt Caching. Claude Platform documentation. https://docs.claude.com/en/docs/build-with-claude/prompt-caching

[7] Joshua Baer. 2026. Agents First (v0.8). https://agentsfirst.dev/

[8] Lisanne Bainbridge. 1983. Ironies of Automation. *Automatica* 19, 6 (1983), 775–779.

Table 9: Four studies would test CLI-first beyond one product.

	Question	Design	Measures
S1	Does the interface change agent outcomes when the product is fixed?	Explored in §6; the confirmatory design above, with injections planted in intermediate data.	Success by whole-state diff, cost by token type, passes, latency, injection success.
S2	Is the CLI a better verification surface for agent-built software?	Agents implement features under CLI acceptance checks, browser end-to-end tests or unit tests only; defects seeded in the core and the GUI; several products.	Defects caught by layer, escaped defects, flake rate, cost, time.
S3	Which CLI design choices matter to agents?	Ablate the command sheet (scope, options, output shapes) and P2–P8 on a fixed product.	Success and cost, especially for small models.
S4	How do people fare when they delegate through agents?	A natural-language front end held constant while the agent's interface and approval granularity vary (per action, per script, out of band); scripted errors and false success reports injected; synthetic data on a sandboxed backend; ethics approval and pre-registration.	Completion, time, detection of injected errors and false reports, understanding of what was done, trust and workload.

[9] Christian Bird, Peter C. Rigby, Earl T. Barr, David J. Hamilton, Daniel M. German, and Prem Devanbu. 2009. The Promises and Perils of Mining Git. In *Proceedings of the 6th IEEE International Working Conference on Mining Software Repositories (MSR)*. 1–10.

[10] Builder.io. 2026. Your App Is Your Agent: Building Agent-Native Applications. https://www.builder.io/blog/your-app-is-your-agent-building-agent-native-applications

[11] Cloudflare. 2025. Code Mode: The Better Way to Use MCP. https://blog.cloudflare.com/code-mode/

[12] Cloudflare. 2026. Introducing cf: The Agentic CLI for the Entire Cloudflare API. https://blog.cloudflare.com/cloudflare-cf-cli-launch/

[13] Alistair Cockburn. 2005. Hexagonal Architecture (Ports and Adapters). https://alistair.cockburn.us/hexagonal-architecture/

[14] Lakshitha de Silva and Dharini Balasubramaniam. 2012. Controlling Software Architecture Erosion: A Survey. *Journal of Systems and Software* 85, 1 (2012), 132–151.

[15] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, et al. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813

[16] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv:2406.13352

[17] Bradley Efron and Robert J. Tibshirani. 1994. *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

[18] Marc Alier Forment, María José Casañ Guerrero, Francisco José García-Peñalvo, and Juanan Pereira. 2026. The Scaffolding Matters More Than the Interface: A Controlled Comparison of MCP and CLI Tool Use Across Seven Agent Scaffoldings, Five Language Models, and One Software Task. arXiv:2608.08654

[19] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173

[20] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[21] Eric Horvitz. 1999. Principles of Mixed-Initiative User Interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99)*. ACM, 159–166. https://doi.org/10.1145/302979.303030

[22] Sunjie Hou, Razvan-Gabriel Dumitru, Edgar Arakelyan, Daniel Yue Zhang, Helen Li, and Chetan Rane. 2026. MCP vs. CLI: Does an AI Agent's Tool Interface Still Matter? Scale Labs blog. https://labs.scale.com/blog/mcp-vs-cli

[23] Cheng-Yu Hsieh, Si-An Chen, Chun-Liang Li, Yasuhisa Fujii, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. 2023. Tool Documentation Enables Zero-Shot Tool-Usage with Large Language Models. arXiv:2308.00675

[24] Edwin L. Hutchins, James D. Hollan, and Donald A. Norman. 1985. Direct Manipulation Interfaces. *Human–Computer Interaction* 1, 4 (1985), 311–338.

[25] Hongrui Jia, Jitong Liao, Xi Zhang, Haiyang Xu, Tianbao Xie, Chaoya Jiang, et al. 2025. OSWorld-MCP: Benchmarking MCP Tool Invocation in Computer-Use Agents. arXiv:2510.24563

[26] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.

[27] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2023. An LLM Compiler for Parallel Function Calling. arXiv:2312.04511

[28] John D. Lee and Katrina A. See. 2004. Trust in Automation: Designing for Appropriate Reliance. *Human Factors* 46, 1 (2004), 50–80.

[29] Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. 2018. NL2Bash: A Corpus and Semantic Parser for Natural Language Interface to the Linux Operating System. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*.

[30] Junting Lu, Zhiyang Zhang, Fangkai Yang, Jue Zhang, Lu Wang, Chao Du, Qingwei Lin, Saravan Rajmohan, et al. 2024. AXIS: Efficient Human-Agent-Computer Interaction with API-First LLM-Based Agents. arXiv:2409.17140

[31] Pattie Maes. 1994. Agents that Reduce Work and Information Overload. *Commun. ACM* 37, 7 (1994), 30–40.

[32] Henry Mao and Rach Pradhan. 2026. MCP vs CLI Is the Wrong Fight. Smithery blog. https://smithery.ai/blog/mcp-vs-cli-is-the-wrong-fight

[33] Waldek Mastykarz. 2026. Don't Rewrite Your CLI for Agents. Microsoft Developer Blog. https://developer.microsoft.com/blog/dont-rewrite-your-cli-for-agents/

[34] M. Douglas McIlroy, Elliot N. Pinson, and Berkley A. Tague. 1978. UNIX Time-Sharing System: Foreword. *The Bell System Technical Journal* 57, 6 (1978), 1899–1904.

[35] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, et al. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. arXiv:2601.11868

[36] MindStudio. 2026. MCP Servers vs CLI Tools for AI Agents: When to Use Each. MindStudio blog, edited by Luis Chavez-Mattos. https://www.mindstudio.ai/blog/mcp-servers-vs-cli-tools-for-ai-agents

[37] Mistral AI. 2026. Spaces: A CLI Built for Humans and Agents. https://mistral.ai/news/spaces

[38] Model Context Protocol. 2025. Model Context Protocol Specification, version 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18

[39] Brad A. Myers and Jeffrey Stylos. 2016. Improving API Usability. *Commun. ACM* 59, 6 (2016), 62–69.

[40] Netlify. 2025. Agent Experience (AX). https://www.netlify.com/agent-experience/

[41] Carman Neustaedter and Phoebe Sengers. 2012. Autobiographical Design in HCI Research: Designing and Learning through Use-It-Yourself. In *Proceedings of the Designing Interactive Systems Conference (DIS)*. 514–523.

[42] Jakob Nielsen. 2023. AI: First New UI Paradigm in 60 Years. Nielsen Norman Group. https://www.nngroup.com/articles/ai-paradigm/

[43] Nordic APIs. 2026. MCP vs. CLI: Which Is Better for Agentic AI? Nordic APIs. https://nordicapis.com/mcp-vs-cli-which-is-better-for-agentic-ai/

[44] Donald A. Norman. 1994. How Might People Interact with Agents. *Commun. ACM* 37, 7 (1994), 68–71.

[45] OWASP Gen AI Security Project. 2025. LLM06:2025 Excessive Agency. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/

[46] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. 2000. A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans* 30, 3 (2000), 286–297.

[47] Dewayne E. Perry and Alexander L. Wolf. 1992. Foundations for the Study of Software Architecture. *ACM SIGSOFT Software Engineering Notes* 17, 4 (1992), 40–52.

[48] Justin Poehnelt. 2026. You Need to Rewrite Your CLI for AI Agents. https://justin.poehnelt.com/posts/rewrite-your-cli-for-ai-agents/

[49] PostHog. 2026. The Golden Rules of Agent-First Product Engineering. https://newsletter.posthog.com/p/the-golden-rules-of-agent-first-product

[50] Aanand Prasad, Ben Firshman, Carl Tashian, and Eva Parish. 2020. Command Line Interface Guidelines. https://clig.dev/

[51] Jia Qi. 2026. MCP vs CLI: Two Ways to Let AI Take Action (and Why It Matters). IBM Community blog. https://community.ibm.com/community/user/blogs/jia-qi/2026/04/08/mcp-vs-cli

[52] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164.

[53] Harini Sampath, Alice Merrick, and Andrew Macvean. 2021. Accessibility of Command Line Interfaces. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI '21)*. ACM. https://doi.org/10.1145/3411764.3445544

[54] Jacob Schmitt. 2026. MCP vs. CLI for AI-native development. CircleCI blog. https://circleci.com/blog/mcp-vs-cli/

[55] Ben Shneiderman and Pattie Maes. 1997. Direct Manipulation vs. Interface Agents. *Interactions* 4, 6 (1997), 42–61. https://doi.org/10.1145/267505.267514

[56] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. arXiv:2402.01030.

[57] Simon Willison. 2025. The Lethal Trifecta for AI Agents: Private Data, Untrusted Content, and External Communication. https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/

[58] Luke Wroblewski. 2009. Mobile First. https://www.lukew.com/ff/entry.asp?933

[59] Luke Wroblewski. 2011. *Mobile First.* A Book Apart, New York, NY, USA.

[60] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. 2023. ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models. arXiv:2305.18323

[61] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2405.15793.

[62] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback. arXiv:2306.14898

[63] J.D. Zamfirescu-Pereira, Richmond Y. Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*.

[64] Mario Zechner. 2025. MCP vs CLI: Benchmarking Tools for Coding Agents. https://mariozechner.at/posts/2025-08-15-mcp-vs-cli/

[65] Chaoyun Zhang, Shilin He, Liqun Li, Si Qin, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. API Agents vs. GUI Agents: Divergence and Convergence. arXiv:2503.11069

[66] Chaoyun Zhang, He Huang, Chiming Ni, Jian Mu, Si Qin, Shilin He, et al. 2025. UFO2: The Desktop AgentOS. arXiv:2504.14603

[67] Xiao Zhou, Siyue Zhang, Yilun Zhao, Jinbiao Wei, Tingyu Song, Arman Cohan, and Chen Zhao. 2026. GUI vs. CLI: Execution Bottlenecks in Screen-Only and Skill-Mediated Computer-Use Agents. arXiv:2606.24551